CMU 11-785

Introduction to Deep Learning

Fall 2026

RECITATION 0.14

TARGET

PSC Bridges-2 / Any Linux HPC

DURATION

~15 min recitation + labs

PREREQS

None — bring curiosity

Introduction to Linux & Infrastructure Essentials

File Systems

Processes

Remote SSH

Shell Automation

Systems Track · Open your SSH session now

TA: Aryan Singh Chandel · Kangping

What We're Covering

01

Core Philosophy

Why Linux is non-negotiable for HPC & DL

02

How Linux Works

Kernel, shell, processes — the mental model

03

File System

FHS structure, navigation, permissions

04

The Pipe Philosophy

Composing commands — Unix's superpower

05

Process Management

Monitor jobs, keep them alive, GPU watch

06

Remote Infrastructure

SSH keys, rsync vs scp, Jupyter tunneling

07

Shell Automation

Compression, pipelines, bash scripting

Why Linux for Deep Learning?

HPC Clusters

Bridges-2 runs headless Linux — no GUI, no mouse. Shell only.

Reproducibility

Exact CUDA pinning, conda envs, deterministic runs require OS control.

Framework Native

PyTorch, JAX, TensorFlow — all designed & optimized on Linux.

```
$ ssh username@bridges2.psc.edu
$ interact -p GPU-shared --gres=gpu:v100-32:1
$ nvidia-smi
```

GPU Direct Access

CUDA stack, kernel scheduler, InfiniBand — all Linux-native.

How Linux Actually Works

Understanding the layers makes every command make sense.

The Kernel

The kernel is the core of Linux. It manages every hardware resource — CPU scheduling, memory allocation, disk I/O, and GPU access. When your Python script calls torch.cuda.is_available() or moves a tensor with .to(device="cuda"), it's ultimately talking to the kernel through the CUDA driver. You never interact with the kernel directly — you go through the shell.

The Shell

The shell (bash) is your command interpreter. Every command you type is forked as a child process. It inherits your environment — $PATH, $HOME, $USER. When you type python3, the shell searches your PATH directories left-to-right and runs the first match it finds.

Everything is a File

Linux treats everything as a file — your GPU is /dev/nvidia0, RAM stats live in /proc/meminfo, network sockets are file descriptors. This abstraction is why pipe (|) works: commands read from stdin and write to stdout, just like reading and writing files.

The Pipe Philosophy

> *“Write programs that do one thing well. Write programs to work together. Write programs that handle text streams.”*
>
> — Doug McIlroy, Bell Labs (inventor of the Unix pipe)

stdin → stdout

Every command reads from stdin, writes to stdout. The pipe (|) connects them. No temp files. Everything flows in memory.

Compose, don't build

You don't need a custom program to find top CPU-hungry Python jobs. You already have ps, grep, sort, head — chain them.

Text is the interface

Because commands exchange plain text, any tool can talk to any other tool. grep doesn't know or care what ps is.

Small tools, big power

ps aux | grep python | sort -k3 -rn | head -5 — four standard tools, zero custom code, exactly the answer you need.

Three Things Worth Understanding

Why rsync, not scp?

scp copies the entire file every time. rsync compares modification time and file size on both sides and only transfers what changed. A 10GB dataset you've already sent? rsync sends nothing. One new checkpoint file added? rsync sends only that file. After the first sync, subsequent runs take seconds.

How SSH keys actually work

You generate a keypair — private key stays on your laptop, public key goes on the server. When you connect, the server sends a random challenge. Only the holder of the private key can answer it correctly. No password crosses the network. Ever. The SSH client refuses to use a private key that other users can read (chmod 600 is required).

nohup vs tmux — when to use which

nohup is a quick fix: it intercepts the hangup signal so your job survives terminal close. But you lose the ability to reconnect and see output live. tmux is the right tool: it creates a session that lives on the remote machine. You detach, log out, come back tomorrow, and reattach to see your training output exactly where you left it.

File System & Navigation

FHS Key Directories

```
/          # Root — everything lives here
├── /home  # Your workspace (quota-limited on HPC)
├── /tmp   # Temp files — wiped on reboot
├── /proc  # Virtual FS — live kernel data
└── /ocean    # PSC Bridges-2 fast storage for datasets
```

Navigation

```
cd ~          # jump to home
cd -          # jump to previous dir
pwd           # print current path
ls -lah       # list all, human sizes
```

Permissions (chmod)

```
chmod 755 script.sh    # rwxr-xr-x
chmod 600 key.pem      # rw------- (private)
chmod +x run.sh        # add execute for all
```

File Operations

```
cp -rv src/ dst/    # recursive copy
mv old new          # move / rename
rm -rf dir/         # no undo!
mkdir -p a/b/c      # nested dirs
find . -name '*.py'
```

Process Management

Monitor

`ps aux | grep python`

Snapshot — find your training PID

`htop`

Interactive, color-coded, kill with F9

`nvidia-smi`

GPU VRAM & utilization live

`kill -15 <PID>`

Polite stop. Use -9 only if unresponsive.

Keep Jobs Alive

```
# nohup — survives terminal close
nohup python train.py > log.txt 2>&1 &
echo "PID: $!"

# tmux — session persists after logout
tmux new -s training
# Ctrl+B then D to detach
tmux attach -t training
```

GPU Monitoring

```
nvidia-smi dmon -s u -d 2
```

Remote Infrastructure

SSH Keys

```
# Generate Ed25519 keypair (local machine)
ssh-keygen -t ed25519 -C "you@cmu.edu"
# Push public key to remote
ssh-copy-id user@bridges2.psc.edu
# ~/.ssh/config shortcut
Host b2
  HostName bridges2.psc.edu
  User jsmith
ssh b2    # one command, done
```

scp vs rsync

```
# scp — copies whole file every time
scp data.zip user@b2:~/data/

# rsync — only sends changed bytes
rsync -avz --progress ./checkpoints/ \
  user@b2:~/project/checkpoints/
```

Jupyter Tunneling

```
# On Bridges-2:
jupyter notebook --no-browser --port=8888
# On local machine:
ssh -N -f -L 8888:localhost:8888 user@b2
# Open localhost:8888 in browser
```

Shell Automation & Pipelines

Compression

```
tar -czf dataset.tar.gz ./dataset/
tar -xzf dataset.tar.gz -C /ocean/
tar -tzf dataset.tar.gz | head -10  # inspect
```

Text Pipelines

```
grep 'val_loss' train.log
ps aux | awk '{print $2,$11}'
sed -i 's/lr=0.01/lr=0.001/g' cfg.yaml

# Chain them:
ps aux | grep python | sort -k3 -rn | head -5
```

Bash Script

```
#!/usr/bin/env bash
set -e          # exit on any error
set -u          # error on unset vars
set -o pipefail # fail if pipe stage fails

PROJECT="$HOME/dl_workspace"

if ! command -v python3 &>/dev/null; then
  echo 'python3 not found' >&2; exit 1
fi

for DIR in data models checkpoints logs; do
  mkdir -p "$PROJECT/$DIR"
done
```

Quick Reference Cheatsheet

Navigation

```
pwd
cd ~/project
ls -lah
find . -name '*.py'
tree -L 2
```

File Ops

```
cp -rv src dst
mv old new
rm -rf dir/
mkdir -p a/b/c
chmod 755 f
```

Processes

```
ps aux | grep py
kill -15 <PID>
nohup cmd &
tmux new -s x
nvidia-smi
```

Remote

```
ssh user@host
scp file u@h:~/
rsync -avz s d
ssh -L 8888:lo:8888
ssh-keygen -t ed25519
```

Archive/Text

```
tar -czf out.tgz ./d
tar -xzf in.tgz
grep -n 'pat' f
awk '{print $2}'
sed 's/a/b/g' f
```

What to Remember From Today

1

The shell is your only interface on HPC

Bridges-2 has no GUI. Everything — launching jobs, moving data, monitoring training — happens through bash. Get comfortable here.

2

nohup saves jobs, tmux saves your sanity

Your SSH session will time out. Use nohup for quick fixes, tmux for anything serious. Set it up before you start training, not after it disconnects.

3

rsync over scp, always

After the first transfer, rsync sends only deltas. For iterative checkpoint syncing, this is the difference between seconds and minutes.

4

set -e, set -u, set -o pipefail

These three lines turn a fragile script into one you can trust. Any script you write for this course should start with all three.

5

Pipes compose power

ps aux | grep python | sort -k3 -rn | head -5 answers a complex question with zero custom code. Learn to think in pipelines.

6

chmod 600 your SSH keys

The SSH client enforces this. Overly open permissions on your private key means connection refused. 600 = owner read/write only.

Hands-On Lab

Run linux_masterclass.sh and dl_setup_pipeline.sh on your Bridges-2 SSH session.

01 SSH into Bridges-2, navigate to scratch without notes

02 Create a full DL project scaffold with mkdir -p in one command

03 Start a job with nohup, verify with ps aux | grep python

04 Run nvidia-smi and identify free VRAM on your GPU node

05 Sync a local directory with rsync — run twice, note the speed

06 Parse a log file with grep + awk to extract epoch and loss

07 Write a 10-line setup script with set -e and a Python version check